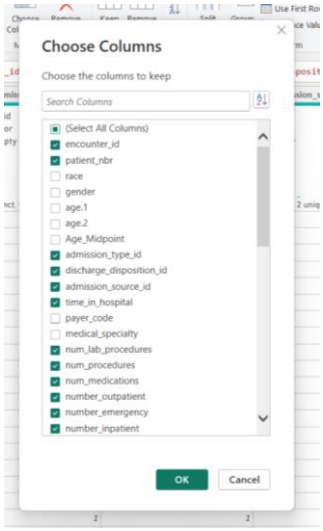
To create the fact table and all dimension tables, I used “**Reference”** and not “**Duplicate**”. This was done to ensure all tables connected to the same original source.

Fact Table Creation (Fact_Encounter):

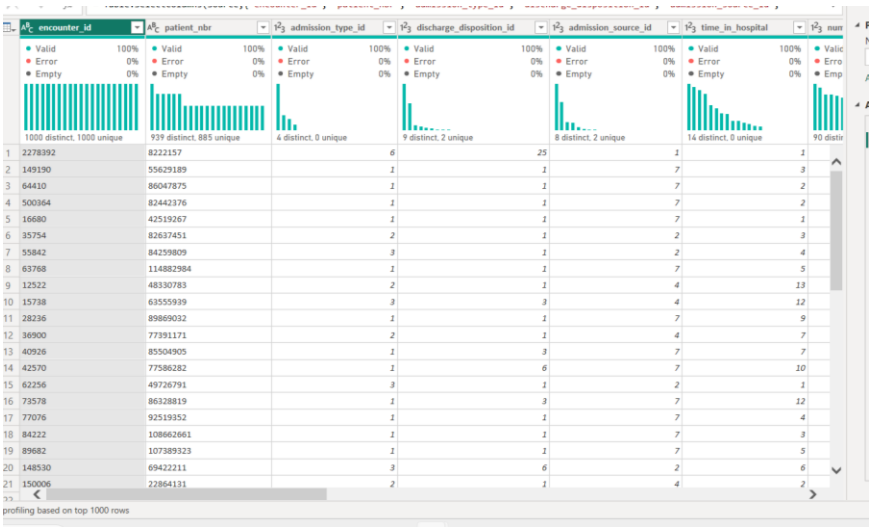
‘**Fact_Encounter’ columns:** ‘*encounter_id (PK)*’, ‘*patient_nbr*’, ‘*admission_type_id*’, ‘*admission_source_id*’, ‘*discharge_disposition_id*’, ‘*diag_1*’ ‘*diag_2*’, ‘*diag_3*’, ‘*time_in_hospital*’, ‘*num_lab_procedures*’, ‘*num_procedures*’, ‘*num_medications*’, ‘*number_outpatient*’, ‘*number_emergency*’, ‘*number_inpatient*’, ‘*number_diagnoses*’, ‘*readmitted*’, ‘*Admission_Key*’, ‘*Diagnosis_Key*’, ‘*Synthetic_Date*’

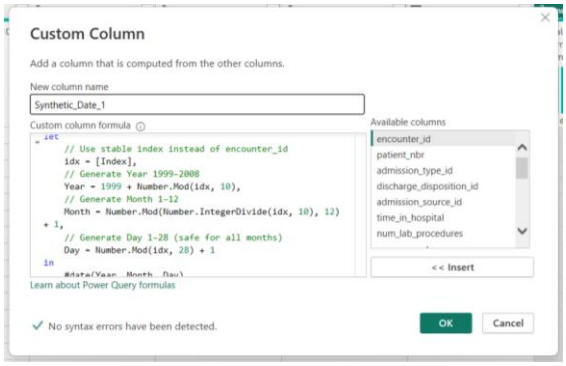
Below is a screenshot of choosing the appropriate columns for ‘Fact_Encounter’ table. I kept numeric/measurable attributes and removed descriptive attributes.

Choosing appropriate columns:



Screenshot of part of the ‘Fact_Encounter’ table after creating it:





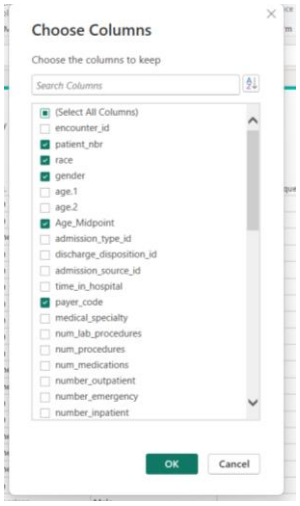
The screenshot on the left is the Synthetic_Date formula. Generating stable random dates using an index. This results to: each encounter getting a unique but stable synthetic date; dates fall between 1999-2008; no row ever changes its date during refresh; avoids duplicates.

Building 'Dim_Patient':

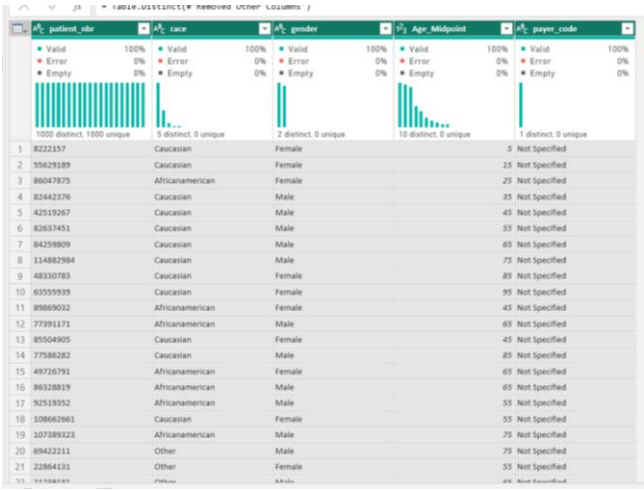
This table contains only patient demographic attributes which include: '*patient_nbr (PK)*', '*race*', '*gender*', '*Age_Midpoint*', '*payer_code*'.

Below is a screenshot of choosing the appropriate columns for 'Dim_Patient' dimension table. Thereafter, I removed duplicate rows to ensure each patient appears once, all demographics are consistent, fact table can safely reference patient_nbr, and there are no broken relationships.

Choosing appropriate columns:



Screenshot the 'Dim_Patient' table after creating it:

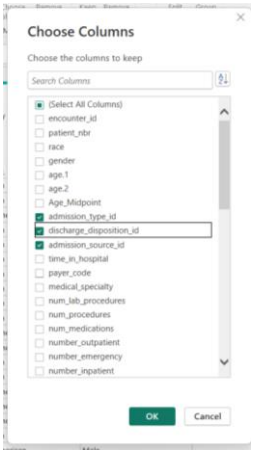


Building 'Dim_Admission':

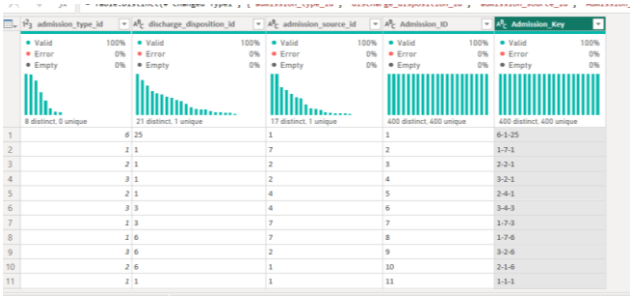
This table categorizes how the patient entered and exited the hospital. Attributes include: '*admission_type_id*', '*admission_source_id*', '*discharge_disposition_id*', '*Admission_ID*', '*Admission_Key*'.

Below is a screenshot of choosing the appropriate columns for 'Dim_Admission' dimension table. Thereafter, I removed duplicate rows, created an index key (Admission_ID), and created a composite key (Admission_Key) to aid while creating a relationship between Fact_Encounter and Dim_Admission.

Choosing appropriate columns:



Screenshot of the 'Dim_Admission' table after creating it:

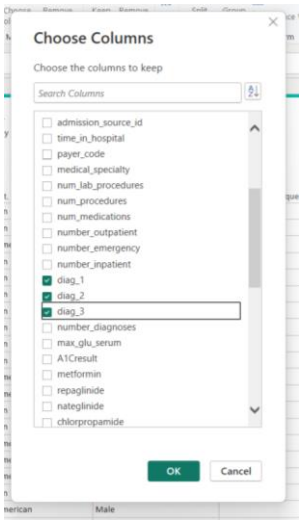


Building 'Dim_Diagnosis':

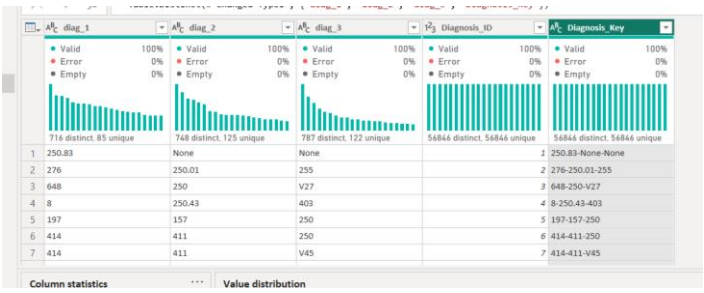
This table has categorical diagnostic labels. Attributes include: '*diag_1*', '*diag_2*', '*diag_3*', '*Diagnosis_ID*', '*Diagnosis_Key*'.

Below is a screenshot of choosing the appropriate columns for 'Dim_Diagnosis' dimension table. Thereafter, I removed duplicate rows, created an index key (Diagnosis_ID), and created a composite key (Diagnosis_Key) to aid while creating a relationship between Fact_Encounter and Dim_Diagnosis.

Choosing appropriate columns:



Screenshot of the 'Dim_Diagnosis' table after creating it:

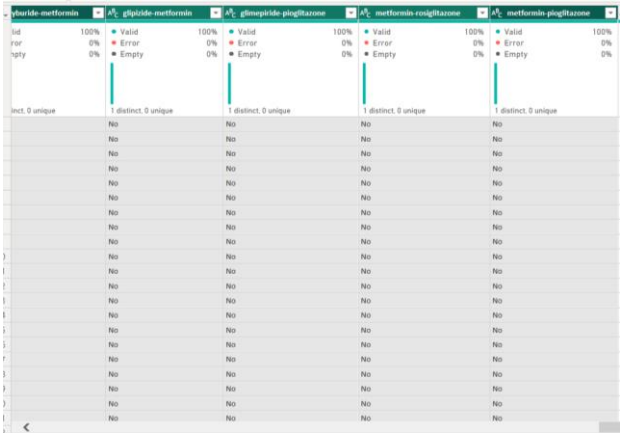


Building 'Dim_Medication':

This table has medication attributes describing which drug and how dosage changed. Attributes include: '*Drug_Name*', '*Dosage_Change_Type*', '*Medication_ID*', '*Medication_Key*'.

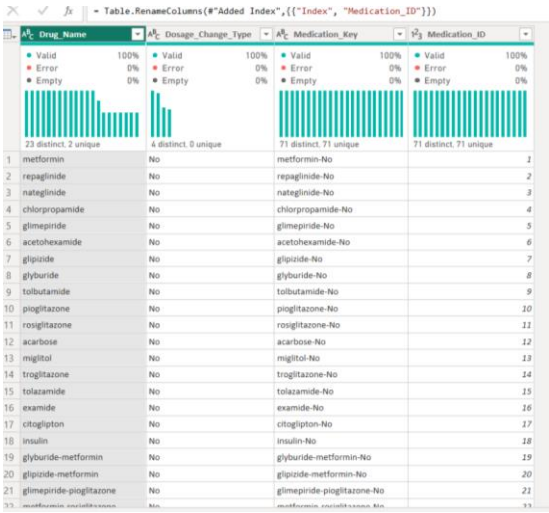
I referenced the cleaned main table and renamed the new table to 'Dim_Medication'. Thereafter, I unpivoted all 24 medication columns to 'Attribute' (which is medication name) and 'Value' (which is dosage change). I renamed 'Attribute' to 'Drug_Name' and 'Value' to 'Dosage_Change_Type'. I then added an index column and renamed it to 'Medication_ID'. Created a composite key 'Medication_Key' to aid when creating a relationship.

Before unpivoting:



After unpivoting:

Screenshot of 'Dim_Medication' table after creating it:



Building 'Dim_Date':

This table is a standard calendar table for time intelligence in DAX. Attributes include: '*Date*', '*Year*', '*Month Number*', '*Month Name*', '*Quarter*', '*Day Name*', '*Date ID*'.

Screenshot of 'Dim_Date' table after creating it:

	Date	Year	Month Number	Month Name	Quarter	Day Name	Date_ID
	Valid 100%	Valid 100%	Valid 100%	Valid 100%	Valid 100%	Valid 100%	Valid
	Error 0%	Error 0%	Error 0%	Error 0%	Error 0%	Error 0%	Error
	Empty 0%	Empty 0%	Empty 0%	Empty 0%	Empty 0%	Empty 0%	Empty
	1000 distinct, 1000 unique	3 distinct, 0 unique	12 distinct, 0 unique	12 distinct, 0 unique	4 distinct, 0 unique	7 distinct, 0 unique	1000 distinct, 10
1	01/01/1999	1999	1	January	1	Friday	
2	02/01/1999	1999	1	January	1	Saturday	
3	03/01/1999	1999	1	January	1	Sunday	
4	04/01/1999	1999	1	January	1	Monday	
5	05/01/1999	1999	1	January	1	Tuesday	
6	06/01/1999	1999	1	January	1	Wednesday	
7	07/01/1999	1999	1	January	1	Thursday	
8	08/01/1999	1999	1	January	1	Friday	
9	09/01/1999	1999	1	January	1	Saturday	
10	10/01/1999	1999	1	January	1	Sunday	
11	11/01/1999	1999	1	January	1	Monday	
12	12/01/1999	1999	1	January	1	Tuesday	
13	13/01/1999	1999	1	January	1	Wednesday	
14	14/01/1999	1999	1	January	1	Thursday	
15	15/01/1999	1999	1	January	1	Friday	
16	16/01/1999	1999	1	January	1	Saturday	
17	17/01/1999	1999	1	January	1	Sunday	
18	18/01/1999	1999	1	January	1	Monday	
19	19/01/1999	1999	1	January	1	Tuesday	
20	20/01/1999	1999	1	January	1	Wednesday	
21	21/01/1999	1999	1	January	1	Thursday	

Formula for creating Dim_Date:

Advanced Editor

Dim_Date

Display Options

```
let
    // Generate 10 years of daily dates (1999-2008)
    Source = List.Dates(
        #date(1999, 1, 1),          // Start date
        3653,                       // 10 years of days
        #duration(1, 0, 0, 0)       // Step = 1 day
    ),

    // Convert list to table
    ToTable = Table.FromList(Source, Splitter.SplitByNothing(), {"Date"}),

    // Add calendar intelligence
    AddYear = Table.AddColumn(ToTable, "Year", each Date.Year([Date]), Int64.Type),
    AddMonthNumber = Table.AddColumn(AddYear, "Month Number", each Date.Month([Date]), Int64.Type),
    AddMonthName = Table.AddColumn(AddMonthNumber, "Month Name", each Date.MonthName([Date]), type text),
    AddQuarter = Table.AddColumn(AddMonthName, "Quarter", each "Q" & Number.ToText(Date.QuarterOfYear([Date])), type text),
    AddDayName = Table.AddColumn(AddQuarter, "Day Name", each Date.DayOfWeekName([Date]), type text),

    // Add surrogate key
    AddIndex = Table.AddIndexColumn(AddDayName, "Date_ID", 1, 1, Int64.Type),
    #"Changed Type" = Table.TransformColumnTypes(AddIndex,{{"Date", type date}})
in
    #"Changed Type"
```

No syntax errors have been detected.

Done Cancel

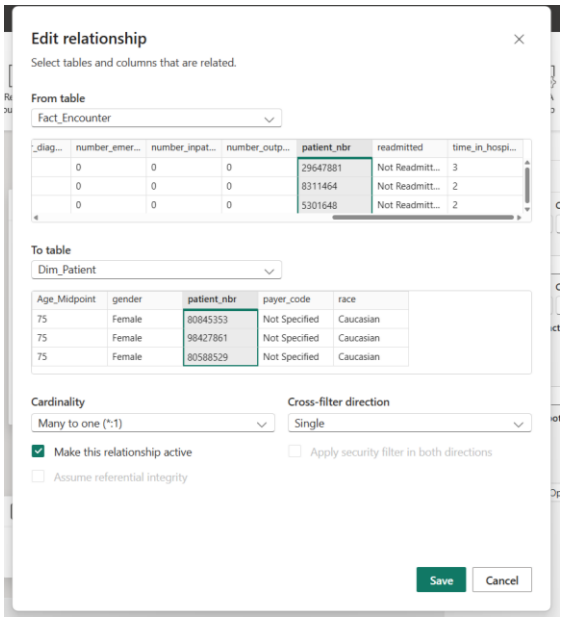
Creating relationships:

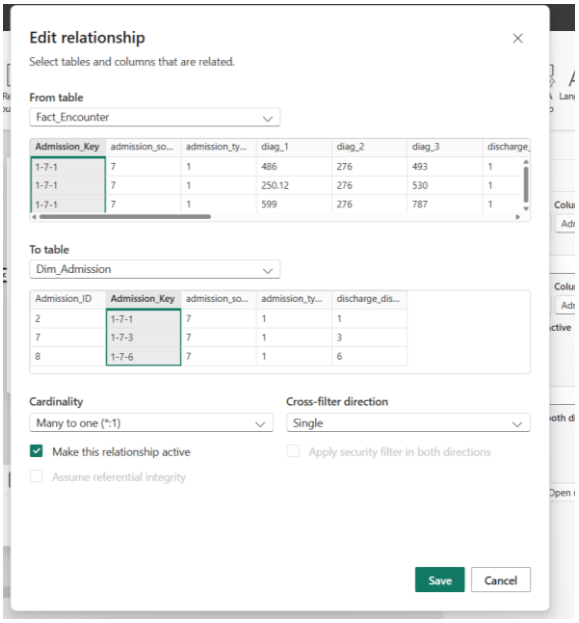
Dim_Patient:

Relationship between Dim_Patient and Fact_Encounter is one-to-many because one patient can have many encounters.

Dim_Admission:

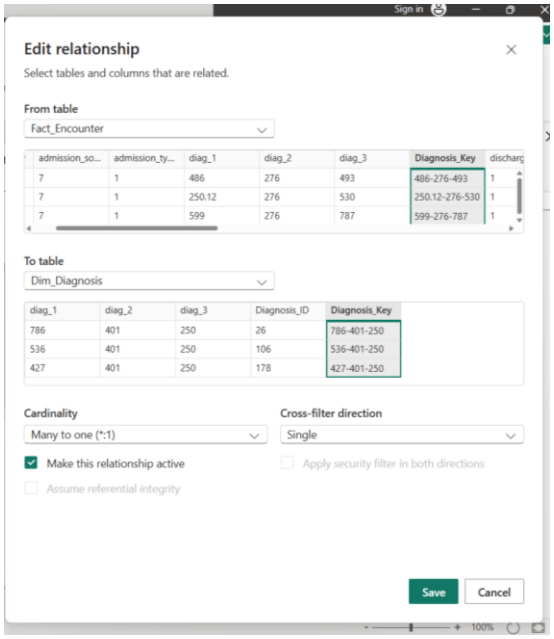
Relationship between Dim_Admission and Fact_Encounter is one-to-many because one admission type can apply to many encounters.





Dim_Diagnosis:

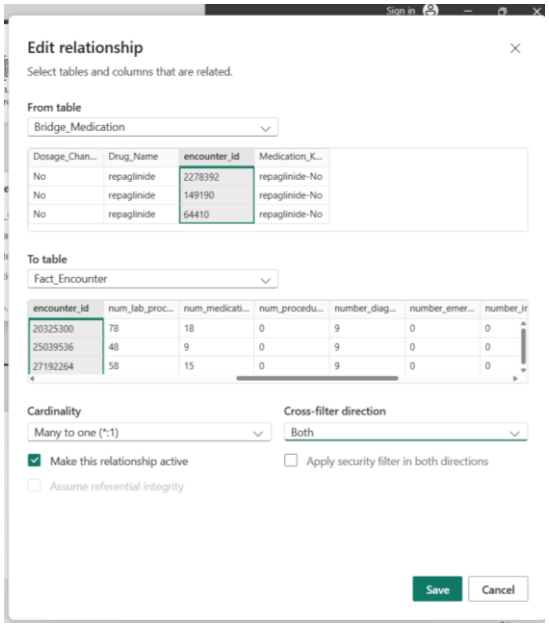
Relationship between Dim_Diagnosis and Fact_Encounter is one-to-many because diagnosis profiles are resued aross encounters.

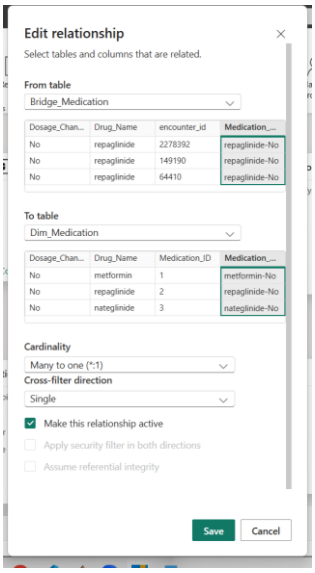


Dim_Medication:

'diabetes_data' contains multiple medication columns per row whereas 'Dim_Medication' contains one medication per row. There grains do no match this can't relate them directly. To create a relationship between these two tables a bridge table (Brirdge_Medication) was created.

Fact_Encounter has a one-to-many relationship with Bridge_Medication (left screenshot). Dim_Medication has a one-to-many relationship with Bridge_Medication (right screenshot).

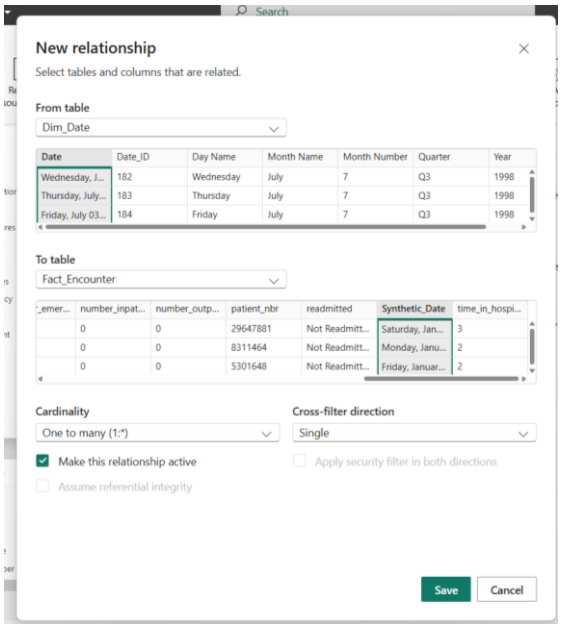




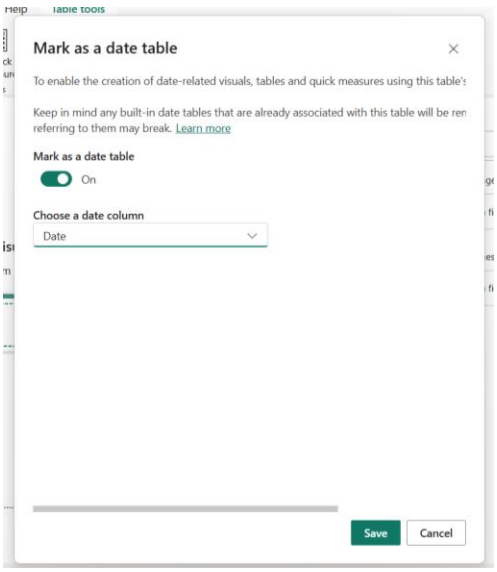
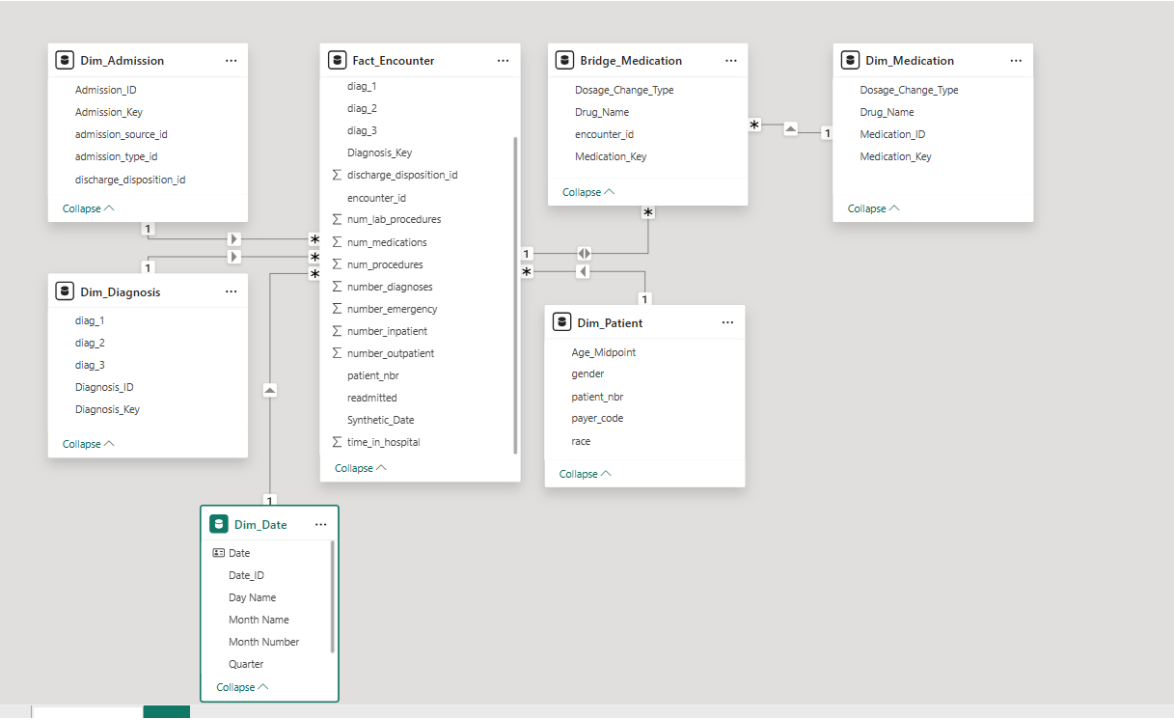
The cross-filter direction is 'Both' for the relationship between Fact_Encounter and Bridge_Medication. This is because it allows the bridge table to communicate with the Fact_Encounter table. Otherwise, if a specific medication is selected in a slicer, the Fact_Encounter may not filter correctly, or the number will stay the same for every drug.

Dim_Date:

Relationship between Dim_Date and Fact_Encounter is one-to-many because daily date dimension supports time intelligence.



Marking Dim_date as date table.



Model View screehshot: